

Techniki Mikroprocesorowe

Sprawozdanie – Laboratorium 5

Temat: System Przerwań Mikrokontrolerów Avr

Autor: Kamil Sitarski

Numer albumu: [REDACTED]

Grupa dziekańska: [REDACTED]

Data wykonania ćwiczenia: 5.05.2026r.

Polecenie 1

1. Opis polecenia

Celem zadania było zaprogramowanie mikrokontrolera ATmega32, aby reagował na sygnały zewnętrzne doprowadzone do linii INT0 oraz INT2. Zadanie wymagało wykorzystania przycisków do generowania przerwań, które miały zmieniać stan diod LED podłączonych do portu A (piny PA0 i PA2). Należało przetestować dwa warianty wyzwalań: zboczem narastającym oraz zboczem opadającym sygnału.

2. Opis kodu

Program realizuje zadanie poprzez wykorzystanie mechanizmu funkcji obsługi przerwań (ISR). Zgodnie z zasadami, pętla główna while(1) jest pusta, a cała logika sterowania odbywa się w procedurach przerwań.

- **Obsługa zdarzeń:**

W programie zdefiniowano dwie funkcje ISR:

```
ISR(INT0_vect) { PORTA ^= 1 << PA0; }  
ISR(INT2_vect) { PORTA ^= 1 << PA2; }
```

Wykorzystują one parametry zwane wektorami przerwań (INT0_vect, INT2_vect), które wskazują mikrokontrolerowi, jaką procedurę wykonać po wystąpieniu zdarzenia na danej linii. Każde wywołanie przerwania powoduje negację stanu odpowiedniego pinu wyjściowego, co w praktyce włącza lub wyłącza diodę LED.

- **Konfiguracja portów i rezystorów podciągających:**

```
DDRA = (1 << PA0) | (1 << PA2); // Ustawienie pinów diod jako wyjścia  
DDRD &= ~(1 << PD2); PORTD |= (1 << PD2); // Konfiguracja PD2 (INT0) z pull-up  
DDRB &= ~(1 << PB2); PORTB |= (1 << PB2); // Konfiguracja PB2 (INT2) z pull-up
```

Piny PA0 i PA2 ustawiono jako wyjścia sterujące diodami LED. Dla linii wejściowych INT0 (PD2) i INT2 (PB2) włączono wewnętrzne rezystory podciągające (pull-up), co zapewnia stabilny stan wysoki na wejściu, gdy przycisk nie jest naciśnięty.

- **Konfiguracja sposobu wyzwalania:** Sposób zgłaszania przerwań zaprogramowano w rejestrach MCUCR (dla INT0) oraz MCUCSR (dla INT2). W wariancie wyzwalania zboczem narastającym (ustawionym w kodzie):

```
tmp = MCUCR; // Odczyt aktualnej zawartości rejestru MCUCR do zmiennej pomocniczej
tmp |= (1 << ISC01) | (1 << ISC00); // Ustawienie bitów ISC01 i ISC00 (zbocze narastające dla INT0)
MCUCR = tmp; // Zapis zmodyfikowanej wartości do rejestru
MCUCSR |= (1 << ISC2); // Ustawienie bitu ISC2 (zbocze narastające dla INT2)
```

Zgodnie z tabelami konfiguracyjnymi, ustawienie obu bitów ISC01 i ISC00 na "1" powoduje reakcję INT0 na zbocze narastające. Podobnie bit ISC2 w rejestrze MCUCSR ustawiony na "1" też aktywuje wyzwalanie INT2 zboczem narastającym.

- **Aktywacja systemu przerwań:**

```
GICR |= (1 << INT0) | (1 << INT2); // Odmaskowanie przerwań
GIFR |= (1 << INTF0) | (1 << INTF2); // Wyczyszczenie flag zgłoszeń
sei(); // Globalne włączenie przerwań
```

Przerwania zostały odmaskowane w rejestrze GICR. Przed startem wyczyszczono również flagi w rejestrze GIFR poprzez zapis jedynki logicznej, co zapobiega wykonaniu przypadkowych przerwań natychmiast po uruchomieniu. Całość systemu aktywowano funkcją sei(), która ustawia bit globalnej obsługi przerwań w procesorze.

3. Kod:

```
/*
 * 05.05.2026r.KS.c
 *
 * Created: 05.05.2026 16:21:22
 * Author : student_pl
 */

#include <avr/io.h>
#include <avr/interrupt.h>

ISR(INT0_vect) {
    PORTA ^= 1 << PA0;
}
ISR(INT2_vect) {
    PORTA ^= 1 << PA2;
}

int main(void) {
    uint8_t tmp;
    DDRA = (1 << PA0) | (1 << PA2);

    DDRB &= ~(1 << PB2); // INT2 jako wejście
    PORTB |= (1 << PB2);

    DDRD &= ~(1 << PD2); // INT0 jako wejście
    PORTD |= (1 << PD2);
    //zbocza opadające
    /*
    tmp = MCUCR;
    tmp |= 1 << ISC01;
    tmp &= ~(1 << ISC00);
    MCUCR = tmp;
    MCUCSR &= ~(1 << ISC2);
    */
    //zbocza narastające
    tmp = MCUCR;
    tmp |= (1 << ISC01) | (1 << ISC00);
    MCUCR = tmp;
    MCUCSR |= (1 << ISC2);
    GICR |= (1 << INT0) | (1 << INT2);
    GIFR |= (1 << INTF0) | (1 << INTF2);
    sei();
    while(1) {}
}
```

Polecenie 2

1. Opis polecenia

Celem zadania było skonfigurowanie 8-bitowego licznika Timer 0 do pracy w trybie CTC (*Clear Timer on Compare Match*). Układ miał odmierzać precyzyjne odcinki czasu, co pozwoliło na cykliczne wywoływanie przerwania. W zadaniu należało uzyskać efekt migania diody LED podłączonej do pinu PA0 z częstotliwością odpowiadającą zliczeniu określonej liczby przerw.

2. Obliczenia

$$F_{CPU} = 1\,000\,000\text{ Hz}$$

$$T_{ustalony} = 100\text{ms} = 0,1\text{s}$$

$$N = 1024\text{ (preskalera)}$$

$$OCR0 = \frac{F_{CPU} \cdot T_{ustalony}}{N} - 1$$

$$OCR0 = \frac{1\,000\,000 \cdot 0,1\text{s}}{N} - 1 \approx 97,6 - 1 \approx 96\text{ (zaokrąglane w dół)}$$

3. Opis Kodu

Program wykorzystuje przerwanie od dopasowania zawartości licznika z zadaną wartością w rejestrze OCR0. Dzięki temu procesor nie musi tracić czasu w pętlach opóźniających (`_delay_ms`), lecz wykonuje zadanie tylko w określonych interwałach.

- **Konfiguracja Timera 0 (tryb CTC i Preskaler 1024):**

```
TCCR0 |= (1 << WGM01) | (1 << CS02) | (1 << CS00); // Ustawienie bitu WGM01 (dla CTC) oraz preskalera 1024
```

```
TCCR0 &= ~(1 << WGM00); // Wyzerowanie bitu WGM00 dla poprawnej pracy w trybie CTC
```

W rejestrze kontrolnym TCCR0 ustawiono bit WGM01 oraz wyzerowano bit WGM00, co aktywuje tryb CTC. W trybie tym licznik TCNT0 zeruje się automatycznie po osiągnięciu wartości zapisanej w rejestrze OCR0. Dodatkowo bity CS02 i CS00 ustawiają preskaler na 1024.

- **Ustawienie progu dopasowania i odmaskowanie przerwania:**

```
OCR0 = 96; // Ustawienie wartości porównania
```

```
TIMSK |= (1 << OCIE0); // Odblokowanie przerwania od dopasowania
```

Wartość w rejestrze OCR0 została ustawiona na 96, co w połączeniu z preskalerem 1024 generuje przerwanie co ok. 100 ms. Rejestr TIMSK z ustawionym bitem OCIE0 odblokowuje lokalne przerwanie od dopasowania (tryb CTC).

- **Obsługa przerwania i licznik programowy:**

```
ISR(TIMERO0_COMP_vect) {  
    static int licznik = 0;  
    licznik++;  
    if(licznik == 10) {  
        PORTA ^= 1 << PA0;  
        licznik = 0;  
    }  
}
```

Zastosowano wektor przerwania TIMERO0_COMP_vect. Ponieważ samo przerwanie występuje co 100 ms, a celem było np. uzyskanie zmiany stanu diody co 1 sekundę, wprowadzono zmienną pomocniczą *static int licznik*. Słowo kluczowe static gwarantuje, że wartość zmiennej jest pamiętana między kolejnymi wywołaniami funkcji ISR. Po zliczeniu 10 wystąpień przerwania ($10 \cdot 100ms = 1s$), stan diody na pinie PA0 jest negowany, a licznik zerowany.

- **Inicjalizacja i pętla główna:**

```
sei();          //globalne odblokowanie przerw  
while(1) {}     //pusta pętla nieskończona, oczekująca na zdarzenia sprzętowe
```

Podobnie jak w poprzednim zadaniu, pętla while(1) pozostaje pusta, co dowodzi, że sterowanie czasem odbywa się w pełni przez system przerw. Globalne włączenie przerw następuje poprzez instrukcję sei().

4. Kod

```
/*
 * 05.05.2026r.KS.c
 *
 * Created: 05.05.2026 16:21:22
 * Author : student_pl
 */

#include <avr/io.h>
#include <avr/interrupt.h>

ISR(TIMER0_COMP_vect) {
    static int licznik = 0;
    licznik++;
    if(licznik==10){
        PORTA ^= 1 << PA0;
        licznik = 0;
    }
}

int main(void) {
    DDRA = (1 << PA0);
    PORTA &= ~(1<<PA0);
    TCCR0 |= (1<<WGM01) | (1<<CS02) | (1<<CS00);
    TCCR0 &= ~(1<<WGM00);
    OCR0 = 96;
    TIMSK |= (1 << OCIE0);
    sei();

    while(1){

    }
}
```

Polecenie 3

1. Opis zadania

Celem zadania było zrealizowanie mechanizmu odczytu stanu przycisków wewnątrz procedury przerwania (co 50 ms) i przesłanie tej informacji do pętli głównej `main()`, która odpowiada za wyświetlenie wyniku na porcie C. Zadanie miało na celu praktyczne wykorzystanie modyfikatora `volatile` oraz zrozumienie roli flag programowych w komunikacji między ISR a resztą programu

2. Obliczenia

$$F_{CPU} = 1\,000\,000\text{ Hz}$$

$$T_{ustalony} = 50\text{ ms} = 0,05\text{ s}$$

$$N = 1024 \text{ (wartość preskalera)}$$

$$OCR0 = \frac{F_{CPU} \cdot T_{ustalony}}{N} - 1$$

$$OCR0 = \frac{1\,000\,000\text{ Hz} \cdot 0,05\text{ s}}{1024} - 1$$

$$OCR0 = \frac{50\,000}{1024} - 1$$

$$OCR0 = 48,82 - 1 \approx 47 \text{ (zaokrąglane w dół)}$$

3. Opis kodu

Program został podzielony na dwie części: sprzętowe zbieranie danych w przerwaniu oraz logiczne przetwarzanie i wyświetlanie w pętli głównej.

- **Zastosowanie modyfikatora volatile:**

```
volatile uint8_t buttons;  
volatile int wsk = 0;
```

Kluczowym elementem są zmienne globalne zadeklarowane ze słowem kluczowym volatile. Informuje ono kompilator, że wartość tych zmiennych może zmienić się w dowolnym momencie (w przerwaniu), przez co program musi zawsze odczytywać je bezpośrednio z pamięci RAM, a nie z rejestrów roboczych procesora. Zapobiega to błędom optymalizacji.

- **Konfiguracja Timera 0 dla 50 ms:**

```
TCCR0 |= (1<<WGM01) | (1<<CS02) | (1<<CS00);  
OCR0 = 47;
```

Licznik ustawiono w tryb CTC z preskalerem 1024. Wartość rejestru OCR0 ustawiona na 47 pozwala na generowanie przerwania co ok. 50 ms. Jest to czas wystarczający do stabilnego odczytu przycisków i eliminacji części drgań styków.

- **Procedura obsługi przerwania (ISR):**

```
ISR(TIMER0_COMP_vect) {  
    buttons = PINA; // Odczyt stanu portu z przyciskami  
    wsk = 1;      // Ustawienie flagi informującej o nowym odczycie  
}
```

W przerwaniu wykonywane są tylko operacje krytyczne czasowo: pobranie stanu fizycznego wejść (PINA) oraz ustawienie flagi pomocniczej wsk. Dzięki temu procedura ISR pozostaje krótka.

- **Pętla główna i mechanizm flagi:**

```
while(1) {  
    if(wsk == 1) {          // Sprawdzenie, czy flaga została ustawiona przez procedurę ISR  
        PORTC = buttons;    // Wyświetlenie stanu na diodach  
        wsk = 0;           // Wyzerowanie flagi przed kolejnym cyklem  
    }  
}
```

W pętli main program stale sprawdza stan flagi wsk. Gdy przerwanie ją ustawi, procesor wykonuje instrukcję wyświetlenia danych na porcie C i natychmiast zeruje flagę, czekając na kolejny cykl timera. Pozwala to odciążać procesor – operacja zapisu na port wykonuje się tylko wtedy, gdy dane faktycznie zostały zaktualizowane.

4. Kod

```
/*
 * 05.05.2026r.KS.c
 *
 * Created: 05.05.2026 16:21:22
 * Author : student_pl
 */

#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint8_t buttons;
volatile int wsk = 0;

ISR(TIMER0_COMP_vect) {
    buttons = PINA;
    wsk = 1;
}

int main(void) {
    DDRA |= 0x00; // Konfiguracja PORTA jako wejścia (domyślnie 0)
    PORTA |= 0xFF; // Włączenie rezystorów podciągających dla wszystkich linii portu A

    DDRC |= 0xFF; // Konfiguracja całego portu C jako wyjścia (wyświetlanie na diodach LED)
    PORTC |= 0x00; // Ustawienie początkowego stanu niskiego na wyjściach portu C

    TCCR0 |= (1<<WGM01) | (1<<CS02) | (1<<CS00);
    TCCR0 &= ~(1<<WGM00);
    OCR0 = 47;
    TIMSK |= (1 << OCIE0);
    sei();

    while(1) {
        if(wsk == 1){
            PORTC = buttons;
            wsk = 0;
        }
    }
}
```

Podsumowanie i wnioski

Realizacja ćwiczeń laboratoryjnych pozwoliła na praktyczne zapoznanie się z mechanizmami systemu przerwań mikrokontrolera ATmega32. Na podstawie przeprowadzonych zadań można wyciągnąć następujące wnioski:

1. Efektywność procesora: Wykorzystanie przerwań pozwala na znaczne zoptymalizowanie pracy procesora. Dzięki przeniesieniu obsługi zdarzeń do funkcji ISR, pętla główna programu może pozostać pusta lub wykonywać inne zadania, zamiast marnować cykle zegara na ciągłe sprawdzanie stanu pinów.
2. Precyzja odmierzenia czasu: Praca z Timerem 0 w trybie CTC udowodniła, że jest to najskuteczniejsza metoda generowania powtarzalnych interwałów czasowych. Pozwala ona uniknąć niedokładności wynikających z używania pętli opóźniających (`_delay_ms`), które są podatne na wstrzymywanie przez inne procesy.
3. Bezpieczeństwo danych i volatile: Ćwiczenie trzecie ukazało rolę modyfikatora volatile. Bez jego zastosowania, optymalizator kompilatora mógłby błędnie założyć, że zmienna w pętli `main()` nie zmienia się, co doprowadziłoby do braku reakcji układu na sygnały z przerwań. Jest to kluczowy aspekt programowania systemów wbudowanych.
4. Konfiguracja sprzętowa: Kluczowe dla poprawnego działania układu było właściwe ustawienie rejestrów sterujących (MCUCR, MCUCSR, TIMSK) oraz globalne odblokowanie przerwań instrukcją `sei()`. Pominięcie któregośkolwiek z tych kroków skutkuje brakiem reakcji mikrokontrolera na zdarzenia.